SHOWSHELF LAB · FOR THE AFTERNOON TEACHER

Teacher guide: Day 3

Students work through the steps in the Lab on their own. Each step has an explanation, a task, two hints and an answer button, so most of the time they can get unstuck alone. This guide gives you every task, a correct answer and the mistakes students usually make, so you can help quickly.

How to help: first ask them to read the "Not yet" message and the red Console line (if any), then open a hint. Only point at the answer after that. In the Lab, **Help › Teacher view** shows the answer and these notes on every step.

Day 3: Lay out the home page

Flexbox, Grid and phones

Yesterday you gave a page colours and fonts. Today you decide **where things go**. You build the layout of the ShowShelf **home page**: a top bar, a search bar and a grid of show cards.

The HTML is ready, and so are the colours, fonts and the look of each card: they're in **base.css** (open its tab above the code box to see it). But base.css has no layout at all, so right now everything is stacked in one long column. Your job is **layout.css**.

You'll use three tools that almost every website uses:

- **Flexbox** to put things in a row: the top bar and the search bar
- **Grid** to arrange the cards in columns
- **Media queries** to show more columns on bigger screens

Your colours from Day 2 are already in the page. Open the **index.html (whole page)** tab any time to see the class names you can use.

Step 1. A centred container ~10 min

Teaches: A `.container` with `max-width`, `margin: 0 auto` and side `padding` centres the page.

TASK

In layout.css, write a `.container` rule with:

1. `max-width: 1200px;`
2. `margin: 0 auto;`
3. `padding: 0 16px;`

COMMON MISTAKES

- `container { }` without the dot.
- `padding: 16px 0` (values swapped): space above and below, but the text still touches the sides.
- `width: 1200px` instead of `max-width`: the page scrolls sideways on a phone.

NOTE

A quick warm-up from Day 2. The preview is narrow, so the centring only shows in the **Bigger** view; the padding is visible straight away.

A CORRECT ANSWER

```
/* layout.css: where things go on the page */

.container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 0 16px;
}
```

Step 2. Flexbox: a row ~15 min

Teaches: `display: flex` on a parent puts its direct children in a row.

TASK

Put the logo and the menu side by side: give `.header-inner` the declaration `display: flex;`.

COMMON MISTAKES

- Putting `display: flex` on the children (`.logo`, `.nav`) instead of the parent.
- Using `.site-header`: its only child is the container, so nothing moves.
- `display: flexbox`.

NOTE

If a student is lost, open the whole page tab together and draw the boxes: header, then container, then logo and nav. "Which box holds the two things we want in a row?"

A CORRECT ANSWER

```
/* ... unchanged ... */
  padding: 0 16px;
}

.header-inner {
  display: flex;
}
```

Step 3. Logo left, menu right ~15 min

Teaches: `justify-content` (along the row) and `align-items` (across it) place flex items.

TASK

In your `.header-inner` rule, add:

1. `justify-content: space-between;` so the logo is on the left and the menu on the right
2. `align-items: center;` so the small menu text lines up with the middle of the bigger logo

COMMON MISTAKES

- `justify-content: centre` or `space between` (with a space).
- Putting the two properties on `.nav` or `.logo` instead of the flex container.
- Mixing up the two: `align-items: space-between`.

A CORRECT ANSWER

```
/* ... unchanged ... */
.header-inner {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

Step 4. Space between the menu links ~10 min

Teaches: `gap` sets the space between flex items. A box can be a flex item and a flex container.

TASK

Write a `.nav` rule that:

1. makes the nav a flex container
2. puts `16px` of `gap` between the links

COMMON MISTAKES

- `gap` on `.nav` without `display: flex`: nothing happens.
- `gap` on `.header-inner`: it only changes the space between the logo and the nav.
- `margin` on every link, which also adds space after the last one.

A CORRECT ANSWER

```
/* ... unchanged ... */
  align-items: center;
}

.nav {
  display: flex;
  gap: 16px;
}
```

Step 5. The hero: centred text ~10 min

Teaches: `text-align: center` on a parent centres all the text inside it.

TASK

Write a `.hero` rule with:

1. `text-align: center;`
2. `padding: 40px 0 28px;` (40px above, 28px below)

COMMON MISTAKES

- `text-align: centre`.
- Centring `h1` and `p` separately (works, but one rule on `.hero` is enough).

A CORRECT ANSWER

```
/* ... unchanged ... */
  gap: 16px;
}

.hero {
  text-align: center;
  padding: 40px 0 28px;
}
```

Step 6. A search bar that stretches ~15 min

Teaches: `flex: 1` on a flex item makes it grow to fill the free space.

TASK

1. Write a `.search` rule with `display: flex;` and `gap: 8px;`.
2. Write a `.search input` rule with `flex: 1;`.
3. In `.search`, add `max-width: 520px;` and `margin: 0 auto;` so the search bar stays a sensible size and centred on big screens.

COMMON MISTAKES

- `flex: 1` on `.search` (the form) instead of on the input.
- `.searchinput` or `.search.input` instead of `.search input` (with a space).
- Forgetting `display: flex` on the form, so `flex: 1` has no effect.

NOTE

Press **Bigger** with them and resize the new window: only the input grows. That picture makes `flex: 1` click.

A CORRECT ANSWER

```
/* ... unchanged ... */
  padding: 40px 0 28px;
}

.search {
  display: flex;
  gap: 8px;
  max-width: 520px;
  margin: 0 auto;
}

.search input {
  flex: 1;
}
```

Step 7. Grid: cards in columns ~20 min

Teaches: `display: grid` with `grid-template-columns: repeat(2, 1fr)` makes two equal columns; `gap` spaces them.

TASK

The six cards are inside `<div class="grid">`. Make it a grid with **2 equal columns** (the right number for a phone) and `16px` of gap.

COMMON MISTAKES

- `repeat(2 1fr)` without the comma, or `repeat(2, 1 fr)` with a space: the line is ignored.
- `grid-template-column` (no s).
- `grid-template-columns: 2;` (a number alone isn't a column size).

NOTE

Good moment to ask: "What would `1fr 2fr` do?" Let them try it, then go back to `repeat(2, 1fr)`.

A CORRECT ANSWER

```
/* ... unchanged ... */
  flex: 1;
}

.grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 16px;
}
```

Step 8. Posters that fill the card ~15 min

Teaches: `width: 100%`, `aspect-ratio: 2 / 3` and `object-fit: cover` give every poster the same shape.

TASK

Write a `.poster` rule with `width: 100%;`, `aspect-ratio: 2 / 3;` and `object-fit: cover;`.

COMMON MISTAKES

- `aspect-ratio: 3 / 2` (a wide picture instead of a tall one).
- `height: 300px` instead of `aspect-ratio`: the shape changes when the column width changes.
- `object-fit: fill` or `contain` (stretched, or empty bars on the sides).

NOTE

The placeholder poster already has a 2:3 shape, so `object-fit` shows no visible change today. Later in the course, with real pictures from TVmaze, it matters. To demo it, temporarily set `aspect-ratio: 1 / 1` and switch `object-fit` between `cover` and `fill`.

A CORRECT ANSWER

```
/* ... unchanged ... */
  gap: 16px;
}

.poster {
  width: 100%;
  aspect-ratio: 2 / 3;
  object-fit: cover;
}
```

Step 9. Phones first, then bigger screens ~20 min

Teaches: Mobile first: phone CSS first, then `@media (min-width: 600px) { ... }` for bigger screens.

TASK

At the bottom of layout.css, add a media query for screens **at least 600px** wide, with a `.grid` rule inside it that makes **3 columns**.

Then press **Bigger** and try it.

COMMON MISTAKES

- `max-width` instead of `min-width`.
- Missing the second `}`, so the rest of the file ends up inside the media query.
- `@media min-width: 600px` without round brackets, or `600` without `px`.
- A property directly inside `@media` without a selector: `@media (...) { grid-template-columns: ...; }`.

NOTE

Students often think their code is broken because the preview doesn't change. It's working: the preview is narrower than 600px. Show them the **Bigger** tab and resizing the window. (If the Lab window itself is small, the preview may be wider and show 3 columns directly.)

A CORRECT ANSWER

```
/* ... unchanged ... */
  object-fit: cover;
}

@media (min-width: 600px) {
  .grid {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

Step 10. A second breakpoint for laptops ~15 min

Teaches: Several breakpoints, from small to big; one media query can hold several rules.

TASK

Under your 600px media query, add one for **900px and wider** with two rules:

1. `.grid` with **4 columns**
2. `.hero h1` with a bigger `font-size`: `2.6rem`

Press **Bigger** and make the window wide to see it.

COMMON MISTAKES

- The 900px query above the 600px one: on a laptop the 600px rule wins, and you get 3 columns.
- The `.hero h1` rule placed after the media query's last `}` (then it applies on phones too).

A CORRECT ANSWER

```
/* ... unchanged ... */
  }
}

@media (min-width: 900px) {
  .grid {
    grid-template-columns: repeat(4, 1fr);
  }
  .hero h1 {
    font-size: 2.6rem;
  }
}
```

Step 11. Cards that lift ~15 min

Teaches: `transform`, `box-shadow` and `transition` on `:hover` make cards lift smoothly.

TASK

1. Write a `.card` rule with `transition: transform 0.15s, box-shadow 0.15s;`.
2. Write a `.card:hover` rule with `transform: translateY(-4px);` and a `box-shadow` like `0 8px 20px rgba(0, 0, 0, 0.12)`.

Move your mouse over the cards in the preview.

COMMON MISTAKES

- `transition` only in `.card:hover`: smooth on the way up, but it jumps back.
- `translateY(4px)` (positive): the card moves down.
- `.card :hover` with a space.

A CORRECT ANSWER

```
/* ... unchanged ... */
  }
}

.card {
  transition: transform 0.15s, box-shadow 0.15s;
}

.card:hover {
  transform: translateY(-4px);
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.12);
}
```

Extra 1. A top bar that stays ~10 min

Teaches: `position: sticky; top: 0;` keeps an element in view while scrolling.

TASK

Make the whole top bar (`.site-header`) sticky at the top. Then scroll the preview to try it.

A CORRECT ANSWER

```
/* ... unchanged ... */
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.12);
}

.site-header {
  position: sticky;
  top: 0;
  z-index: 10;
}
```

Extra 2. Six columns on big screens ~10 min

Teaches: A 1200px breakpoint for big screens, placed last.

TASK

Add a media query for `min-width: 1200px` that gives the grid **6 columns**. Check it with **Bigger** on a wide window.

A CORRECT ANSWER

```
/* ... unchanged ... */
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.12);
}

@media (min-width: 1200px) {
  .grid {
    grid-template-columns: repeat(6, 1fr);
  }
}
```

Extra 3. A grid without media queries ~15 min

Teaches: `repeat(auto-fill, minmax(160px, 1fr))` fits as many columns as there's room for.

TASK

1. In your normal `.grid` rule, replace `repeat(2, 1fr)` with `repeat(auto-fill, minmax(160px, 1fr))`.
2. Delete the `.grid` rules inside your media queries (they would overrule it). Keep the `.hero h1` rule.

Try it with **Bigger** and resize the window: the columns appear one by one.

A CORRECT ANSWER

```
/* ... unchanged ... */
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(160px, 1fr));
  gap: 16px;
}
/* ... unchanged ... */



@media (min-width: 900px) {
  .hero h1 {
/* ... unchanged ... */
```